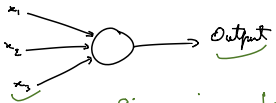


Neural Network and Deep Learning

Michael Nielson

Perceptrons:



Binary in and out

Simple Rule: weights representing importance of respective inputs to the output.

Output determined by $\sum w_i x_i$ as the following:

$$\text{output} = \begin{cases} 0 & \text{if } \sum w_i x_i \leq \text{threshold} \\ 1 & \text{if } \sum w_i x_i > \text{threshold} \end{cases}$$

To simplify

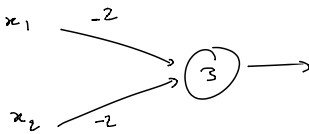
• Use Dot Product: $w \cdot x = \sum w_i x_i$ (w, x weight and input vectors)

• Define b : $b \equiv -\text{threshold}$ (moves it to the other side of the algebraic expression.)
(bias)

Then

$$\text{output} = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

Test example:



or and **NAND**

0 0 1 $x_1=0$
 $x_2=0$ $0+0+3=3 > 0 \rightarrow \text{Out}=1$

1 0 1 $x_1=0$
 $x_2=1$ $0-2+3=1 > 0 \rightarrow \text{Out}=1$

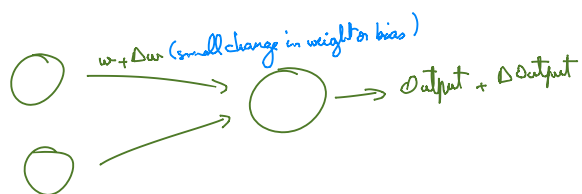
1 1 1 $x_1=1$
 $x_2=0$ $-2+0+3=1 > 0 \rightarrow \text{Out}=1$

1 1 0 $x_1=1$
 $x_2=1$ $-2-2+3=-1 < 0 \rightarrow \text{Out}=0$

Our Perceptron implements a **NAND** gate

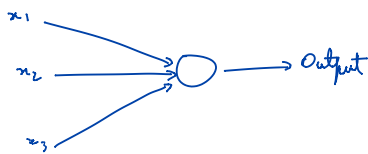
Sigmoid Neurons:

A learning system is when we make a slight change in the weights so that we can get a slight change in output, this change will make the network learn and have closer future choices.



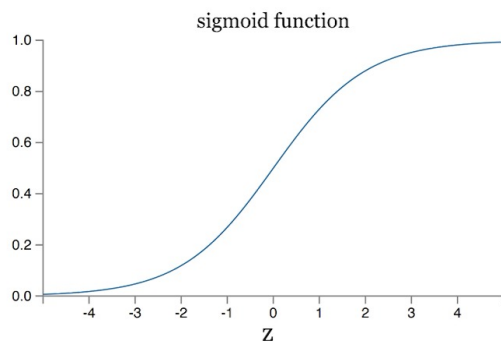
But with a normal perceptron, this can't be done since the output is either 1 or 0, thus the result stays the same or change drastically.

→ For this we introduce sigmoid neuron



The diff is x_i 's can be any number between 0 and 1, it also have weights $w_1, w_2, \dots$, and a bias b .

and this time the output is $\sigma(w \cdot x + b)$ " σ : sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ "



We have an approximation of the slight change:

$$\Delta \text{output} = \frac{d\sigma_{\text{output}}}{dw_j} \Delta w_j + \frac{d\sigma_{\text{output}}}{db} \Delta b$$

Exercise:

Suppose we take all the weights and biases in a network of perceptrons and multiply them by a positive constant $c > 0$, show that the behaviour of network doesn't change.

Suppose a perceptron network, there are two outputs based on the following:

$$\text{output} = 0 \text{ if } \sum x_j w_j + b \leq 0$$

$$\text{output} = 1 \text{ if } \sum x_j w_j + b > 0$$

Now considering new weights and bias such that $w'_j = c w_j$ and $b' = c b$ ($c > 0$)

$$\text{then } \sum x_j w'_j + b' = \sum x_j c w_j + c b = c [\sum x_j w_j + b] \leq 0$$

since $c > 0$, the sign will not change

Therefore same output, thus the behaviour of the network doesn't change.

• Sigmoid neurons simulating perceptrons, part II

Suppose we have the same setup as the last problem - a network of perceptrons. Suppose also that the overall input to the network of perceptrons has been chosen. We won't need the actual input value, we just need the input to have been fixed. Suppose the weights and biases are such that $w \cdot x + b \neq 0$ for the input x to any particular perceptron in the network. Now replace all the perceptrons in the network by sigmoid neurons, and multiply the weights and biases by a positive constant $c > 0$. Show that in the limit as $c \rightarrow \infty$ the behaviour of this network of sigmoid neurons is exactly the same as the network of perceptrons. How can this fail when $w \cdot x + b = 0$ for one of the perceptrons?

when we change to sigmoid neurons, we will have

$$\text{output} = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-w \cdot x + b}}$$

then applying the changes to weights and bias:

$$\sigma(w'x + b') = \frac{1}{1 + e^{-c(w \cdot x + b)}}$$

$$c > 0 \text{ and } c \rightarrow \infty$$

$$\text{if } w \cdot x + b > 0 \Rightarrow \sigma(w'x + b') = 1$$

$$\text{if } w \cdot x + b < 0 \Rightarrow \sigma(w'x + b') = 0$$

these are the same as the perceptron example.

Now for $w \cdot x + b = 0$

$$\Rightarrow \sigma(w \cdot x + b) = \frac{1}{1 + e^{-0}} = \frac{1}{2} \text{ (not an acceptable result) fails}$$

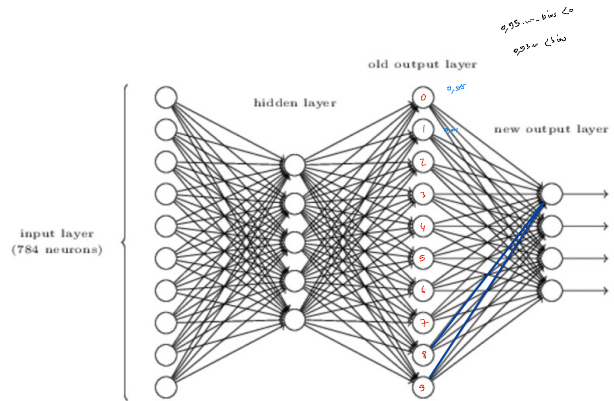
The architecture of Neural Networks

- Till now we have only seen networks called **feed-forward** where the output from one layer is used as input to the next layer, there are no loops. info is always fed forward not back → here comes the **recurrent neural network**

* A Simple Network to classify handwritten digits

Exercise

- There is a way of determining the bitwise representation of a digit by adding an extra layer to the three-layer network above. The extra layer converts the output from the previous layer into a binary representation, as illustrated in the figure below. Find a set of weights and biases for the new output layer. Assume that the first 3 layers of neurons are such that the correct output in the third layer (i.e., the old output layer) has



Learning with Gradient Descent:

We will be using the MNIST data to train and test

Let's denote x as the input " $28 \times 28 = 784$ dimensional vector", and the output $y = y(x)$ a 10 dimensional vector. $y(x) = (0, 0, \dots, 1, 0)^T$

- And now we define a **cost function** that determines how good/bad our weight/bias choice was

$$C(w, b) = \frac{1}{2n} \sum_x \|y(x) - a\|^2$$

